

수능 수학 풀이 코치 앱 구조 설명서

아이디어, 코칭 설계, 그래프 결정 로직, SW 구조를 풀어서 설명한 전달용 문서

이 앱은 정답을 빠르게 출력하는 앱이 아니라, 학생이 막힌 자리에서 다시 한 줄을 쓰게 만드는 풀이 코치입니다. 특히 그래프는 보기 좋게 넣은 장식이 아니라, 학생이 지금 봐야 할 수학적 관계를 고르는 표현 방식입니다. 이 문서는 앱이 어떤 문제의식에서 출발했고, 그 생각이 어떤 화면 구조와 코드 구조로 바뀌었는지를 풀어서 설명합니다.

1. 처음 문제의식

처음 출발점은 고3 미적분 학습 부진이었습니다. 치환적분, 정적분, 급수와 정적분의 관계처럼 여러 개념이 함께 나오는 단원에서는 학생이 계산을 몰라서만 막히는 것이 아닙니다. 더 자주 나타나는 문제는 “무엇을 먼저 봐야 하는지”를 놓치는 것입니다. 긴 수식이 나오면 학생은 식 전체를 한꺼번에 처리하려고 하고, 그 순간 풀이의 첫 줄을 쓰기 전에 멈추는 경우가 많습니다.

그래서 이 앱은 완성 풀이를 대신 써 주는 방향으로 설계하지 않았습니다. 완성 풀이를 바로 보여 주면 학생은 잠깐 안심할 수 있지만, 다음 문제에서 다시 같은 위치에 멈출 가능성이 큼니다. 이 앱의 목표는 학생이 지금 있는 위치를 확인하고, 바로 다음에 할 행동 하나를 스스로 시도하게 만드는 것입니다.

학습 문제

긴 수식, 낯선 그래프, 여러 조건이 한꺼번에 나오면 학생은 계산 전에 먼저 멈춥니다.

설계 목표

정답보다 “지금 확인할 대상 하나”를 먼저 잡아 주고, 학생이 다음 줄을 직접 써 보게 합니다.

구현 방향

문제 입력, AI 코칭, 그래프 표현, 오답 기록, 배포 구조를 하나의 웹앱 흐름으로 연결했습니다.

2. 코칭 구조를 이렇게 나눈 이유

학생이 막혔을 때 필요한 것은 긴 해설보다 현재 위치를 다시 잡아 주는 짧은 안내입니다. 그래서 응답은 여러 조각으로 나누었습니다. 먼저 AI가 학생의 입력을 어떤 수식으로 읽었는지 보여 주고, 그다음 지금 막힌 핵심을 짚고, 바로 해 볼 다음 행동을 제시합니다. 정답은 바로 보이지 않게 접어 두고, 힌트는 약한 힌트부터 강한 힌트까지 단계적으로 열 수 있게 했습니다.

구성	역할	그렇게 나눈 이유
read	학생 입력을 AI가 어떻게 읽었는지 보여 줍니다.	사진이나 수식이 잘못 읽히면 이후 코칭도 틀어지므로, 가장 먼저 확인해야 합니다.
diagnosis	지금 막힌 핵심을 한 문장으로 짚습니다.	학생이 전체 문제 속에서 어디에 서 있는지 알게 하기 위한 장치입니다.
next_step	바로 해 볼 행동 하나를 제시합니다.	“증감표를 완성해 봐”가 아니라 “대표값 하나를 넣어 봐”처럼 행동을 작게 만듭니다.
hints	약/중/강 힌트를 단계적으로 공개합니다.	처음부터 많은 정보를 주면 다시 해설을 받아 적는 흐름으로 돌아가기 때문입니다.
viz	필요한 경우 그래프 사양을 제공합니다.	수식이 뜻하는 관계를 눈으로 확인하게 하되, 정답을 직접 드러내지 않게 합니다.
answer	정 막혔을 때 볼 최종 방향을 담습니다.	학습 흐름을 지키기 위해 정답은 기본 화면에서 분리했습니다.

3. SW 구조: 선택과 렌더링을 분리함

이 앱에서 중요한 개발 판단은 AI와 화면의 역할을 섞지 않는 것이었습니다. 브라우저가 직접 API 키를 들고 있으면 보안상 문제가 생기고, AI가 긴 문장만 돌려주면 화면이 그 응답을 안정적으로 나누어 보여 줄 수 없습니다. 그래서 서버리스 함수는 AI 호출과 코칭 기준을 맡고, 브라우저는 입력 처리와 화면 렌더링을 맡도록 나누었습니다.

파일	맡은 일	설계 이유
index.html	문제 입력, 결과, 그래프, 이어가기 입력칸의 화면 위치를 잡습니다.	화면 뼈대와 동작 로직을 분리해 수정 위치를 분명하게 합니다.
style.css	카드형 UI, 반응형 배치, 버튼과 입력창의 상태를 정리합니다.	학습 도구는 반복해서 읽는 화면이므로 과한 장식보다 가독성이 중요합니다.
app.js	사진 압축, 대화 누적, 요청, 코칭 렌더링, 그래프 표시, 실수 기록을 처리합니다.	학생과 바로 상호작용하는 작업은 브라우저에서 빠르게 처리해야 합니다.
api/coach.js	Gemini 호출, API 키 보호, 튜터 기준, 그래프 결정 기준을 담당합니다.	외부 AI 호출과 보안 정보는 서버 쪽에 두어야 안전합니다.

핵심은 `app.js` 가 수학 문제를 다시 푸는 규칙 엔진이 아니라는 점입니다. 수학적 판단은 `api/coach.js` 의 튜터 기준을 통해 `viz` JSON으로 정리되고, 브라우저는 그 JSON을 정해진 방식으로 그립니다.

4. 그래프 선택을 로직으로 본 이유

그래프는 “수학 앱처럼 보이게 하려고 넣은 그림”이 아닙니다. 수학 문제에서 그림은 학생이 식을 이해하는 방식 자체를 바꿀 수 있습니다. 하지만 아무 문제에도 그래프를 붙이면 오히려 학습을 방해합니다. 경우의 수 문제에서 막대 그림을 억지로 그리거나, 단순 대입 계산만 남은 상황에서 함수 그래프를 보여 주면 학생은 정작 해야 할 행동에서 멀어질 수 있습니다.

그래서 이 앱의 그래프 선택은 먼저 “그림이 필요한가?”를 묻습니다. 필요하다면 그 다음에는 “무엇을 보여 줘야 하는가?”를 묻습니다. 함수의 위아래 관계인지, 도형의 보조선인지, 원이나 타원 같은 곡선 자체인지, 회전체처럼 입체 구조인지에 따라 그래프 종류가 달라집니다. 마지막으로 “이 그림이 정답을 먼저 말해 버리지 않는가?”를 검사합니다.

그래프 선택의 핵심 순서

1. next_step을 먼저 정한다.
예: "x=1을 넣어 어느 함수가 위인지 비교해볼래?"

2. 그 next_step을 돕는 그림이 필요한지 판단한다.
필요 없으면 {"kind": "none"} 으로 둔다.

3. 수학 대상의 형태를 나눈다.
함수 관계 / 도형 관계 / 음함수 / 매개곡선 / 3D 구조 / 공간 점 관계

4. 정답 유출 위험을 확인한다.
점, 라벨, 제목, 중심, 반지름, 최종값이 답을 먼저 말하지 않는지 본다.

5. 브라우저가 계산 가능한 JSON으로 바꾼다.
kind에 맞는 수식, 범위, 점, 선분, 음영 정보를 넣는다.

5. 그래프 종류별 판단 기준

아래 표는 그래프 종류를 단순히 나열한 것이 아니라, 어떤 수학적 관계를 보여 주기 위해 그 종류를 선택하는지를 정리한 것입니다. 같은 “함수” 문제라도 넓이 문제인지, 극값 판별인지, 단순 계산인지에 따라 그래프가 달라집니다.

판단 질문	선택	보여 주는 관계	주의점
함수의 변화, 교점, 위아래, 넓이가 핵심인가?	function2d	곡선의 위치, 교점, 구간, 음영을 보여 줍니다.	넓이값이나 극값 자체는 라벨에 쓰지 않습니다.
점, 선분, 삼각형, 원, 접선, 보조선 관계가 핵심인가?	geometry2d	좌표평면 위의 도형 관계와 작도 방향을 보여 줍니다.	중심·반지름이 답이면 완성된 원보다 작도선을 먼저 보여 줍니다.
$y=f(x)$ 보다 $F(x,y)=0$ 형태가 자연스러운가?	implicit2d	원, 타원, 쌍곡선, 음함수의 모양을 보여 줍니다.	도형 작도 관계가 핵심이면 geometry2d 가 우선입니다.
$x(t)$, $y(t)$ 처럼 매개변수로 움직이는가?	parametric2d	매개변수 변화에 따라 곡선이 생기는 흐름을 보여 줍니다.	t 가 실제 매개변수일 때만 남깁니다.
단면을 축 둘레로 돌리는가?	solid_revolution	회전해서 생기는 입체를 보여 줍니다.	부피값은 쓰지 않고 회전 구조만 보여 줍니다.
두 변수 함수의 높이 변화가 핵심인가?	surface3d	$z=f(x,y)$ 곡면을 보여 줍니다.	미정 문자를 남기면 곡면이 사라질 수 있습니다.
공간의 점, 벡터, 거리 관계가 핵심인가?	points3d	점과 선분을 실제 공간 위치에 놓습니다.	곡면이 필요한 상황과 구분합니다.
그림이 오히려 판단을 흐리는가?	none	그래프를 만들지 않습니다.	경우의 수, 단순 대입, 계산만 남은 상황이 여기에 해당합니다.

6. 판단 예시: 넓이 문제

예를 들어 두 곡선 사이 넓이 문제에서 학생이 교점은 구했지만 적분식을 세우지 못했다고 하겠습니다. 이때 학생에게 필요한 것은 적분값이 아니라 “어느 함수가 위에 있는가”입니다. 그래서 그래프는 최종 넓이값을 보여 주지 않고, 두 곡선과 구간 음영만 보여 줍니다.

학생의 막힘:
교점은 구했는데 넓이 적분식을 어떻게 세우는지 모르겠어.

next_step:
x=1을 넣어 두 함수 중 어느 쪽이 위인지 비교해볼래?

판단:
그래프 필요 = 예
수학 대상 = 평면 함수 두 개의 위아래 관계
정답 유출 위험 = 넓이값을 직접 쓰면 안 됨
선택 = function2d + shade

이때 viz 에는 두 함수의 식, x축 범위, 음영을 칠할 구간이 들어갑니다. 음영은 학생이 넓이 구간을 눈으로 확인하도록 돕지만, 넓이를 계산한 최종값은 보여 주지 않습니다.

viz 핵심 사양
kind: function2d
curves: $y=x$, $y=x^2-2x$
xRange: [-1, 4]
shade: from 0 to 3, lower x^2-2x , upper x

7. 판단 예시: 극값 문제

극값 문제에서는 더 조심해야 합니다. 학생이 $f'(x)=0$ 후보까지 구했지만 어느 쪽이 극대인지 극소인지 모르는 상황을 생각해 보겠습니다. 겉으로는 원래 함수 그래프를 그려 주는 것이 좋아 보일 수 있습니다. 그러나 원래 함수 위에 후보점을 찍으면 그 점의 y 값이 곧 극댓값이나 극솟값이 됩니다. 그러면 그래프가 힌트가 아니라 정답을 먼저 말하는 장치가 됩니다.

그래서 이 앱은 극값 판별 상황에서는 원래 함수가 아니라 도함수 그래프를 우선합니다. 도함수 그래프 위에서 $f'(x)=0$ 후보 위치만 보여 주고, 학생이 왼쪽 또는 오른쪽 대표값을 넣어 부호 변화를 직접 확인하도록 유도합니다.

학생의 막힘:

후보는 두 개 나왔는데 어느 쪽이 극대인지 극소인지 모르겠어.

next_step:

$x=-1$ 의 왼쪽 대표값 -2 를 $f'(x)$ 에 넣어 부호만 볼래?

판단:

그래프 필요 = 예

수학 대상 = 함수 변화

정답 유출 위험 = 원래 함수 위 점의 y 값이 답이 될 수 있음

선택 = 도함수의 function2d

viz 핵심 사양

kind: function2d

curve: $f'(x)=3x^2-3$

xRange: $[-3, 3]$

yRange: $[-5, 15]$

points: $(-1, 0)$, $(1, 0)$, label "후보점"

여기서 중요한 것은 “후보점”이라는 라벨도 조심해서 써야 한다는 점입니다. “극대 후보”, “극소 후보”처럼 방향을 알려 주는 라벨은 학생의 판단을 대신할 수 있으므로 피합니다.

8. 판단 예시: 원, 외접원, 타원

그래프 로직에서 가장 헷갈리기 쉬운 부분은 같은 단어가 나와도 실제로 보여 줘야 하는 것이 달라진다는 점입니다. 예를 들어 “원”이라는 단어가 나온다고 해서 항상 같은 그래프를 쓰면 안 됩니다. 원의 방정식 자체가 핵심이면 음함수 그래프가 자연스럽게, 삼각형의 외접원이나 접선 문제라면 함수 그래프보다 도형 관계가 중요합니다.

겉으로 보이는 말	실제로 봐야 할 것	선택	이유
원	원의 방정식 자체	implicit2d	$F(x,y)=0$ 곡선 자체가 핵심입니다.
원	외접원, 접선, 수직이등분선	geometry2d	점, 선분, 보조선 관계가 풀이의 핵심입니다.
타원	$x^2/25+y^2/9=1$ 의 모양	implicit2d	$y=f(x)$ 로 나누면 위아래 식이 복잡해집니다.

외접원 문제에서는 중심과 반지름을 바로 계산해 보여 주는 것이 항상 좋은 설명은 아닙니다. 중심이나 반지름이 최종 답에 가까운 경우에는 완성된 원보다 중점, 수직이등분선, 후보선 같은 작도 방향을 먼저 보여 주는 편이 학습 목적에 맞습니다.

외접원 보조선의 viz 핵심 사양

kind: geometry2d

points: A, B, C, AB 중점

segments: AB, BC, CA

lines: AB의 수직이등분선
circles: 정답 유출 위험이 있으면 생략 가능

타원 방정식의 viz 핵심 사양
kind: implicit2d
expr: $x^2/25 + y^2/9 - 1$
xRange: [-6, 6]
yRange: [-4, 4]

9. 판단 예시: 3D와 none

3D 그래프는 멋있어 보이기 때문에 무조건 쓰고 싶어질 수 있지만, 이 앱에서는 3D도 이유가 있을 때만 사용합니다. 회전체 부피처럼 “단면을 돌린다”는 감각이 핵심이면 3D가 도움이 됩니다. 공간좌표 문제에서 두 점 사이 거리나 벡터 방향을 볼 때도 3D 점과 선분이 의미가 있습니다. 반대로 경우의 수처럼 선택 구조가 핵심인 문제는 3D는 물론 2D 그림도 만들지 않습니다.

상황	선택	왜 그렇게 고르는가
$y=\sqrt{x}$ 를 x축 둘레로 회전	solid_revolution	반지름이 회전해 원판 단면을 만든다는 구조가 핵심입니다.
$z=f(x,y)$ 형태의 두 변수 함수	surface3d	높이 변화가 평면보다 3D 곡면에서 더 직접적으로 보입니다.
공간좌표의 두 점 사이 거리	points3d	점과 선분 위치 관계가 핵심이므로 공간에 점을 놓아야 합니다.
서로 다른 공을 사람에게 나누기	none	각 공의 선택지 수와 공의 법칙이 핵심이라 그래프가 필요 없습니다.

10. 렌더링 가능성 검사

AI가 좋은 그래프 사양을 만들더라도, 브라우저가 실제로 계산할 수 없으면 화면에는 아무것도 나오지 않습니다. 그래서 수학과 프로그래밍 식 사이의 차이를 따로 고려했습니다. 예를 들어 x^2+a 처럼 값이 정해지지 않은 문자가 남아 있으면 그래프 계산이 실패합니다. 또 $\sqrt{\text{음수}}$, $\log(0)$, $1/0$ 같은 값은 정의역 밖이므로 전체 그래프를 깨뜨릴 수 있습니다.

검사 항목	문제	처리 방식
미정 문자	값이 정해지지 않은 a , k , m 이 남으면 곡선이 사라질 수 있습니다.	조건상 값이 정해지면 숫자로 넣고, 아니면 대표값을 쓰거나 곡선을 뺍니다.
허용 변수	그래프 종류마다 계산 가능한 변수가 다릅니다.	function2d 는 x , implicit2d 는 x,y , 매개곡선은 t 를 사용합니다.
정의역 오류	일부 점에서 계산이 불가능하면 전체 그래프가 깨질 수 있습니다.	점별로 안전 평가해 실패한 지점만 끊어 그립니다.
정답 라벨	“정답”, “극댓값”, “넓이값” 같은 표현은 힌트가 아니라 답지가 됩니다.	튜터 기준에서 금지하고, 브라우저에서도 중립 표현으로 완화합니다.

11. 화면에서 실제로 그려지는 과정

app.js 는 AI가 보낸 viz 객체를 보고 그래프 종류별 렌더러를 실행합니다. 예를 들어 function2d 이면 x 값 배열을 만들고, 각 x 값에 대해 수식을 계산한 뒤 Plotly 선 그래프로 그립니다. geometry2d 이면 점, 선분, 다각형, 보조선을 각각 Plotly trace로 바꿉니다. 3D 계열은 Plotly의 surface나 scatter3d trace를 사용합니다.

app.js 처리 흐름

1. AI 응답에서 viz.kind 확인
2. kind가 none이면 그래프 카드 숨김
3. function2d이면 xRange로 x값 배열 생성
4. math.js가 expr를 계산
5. shade가 있으면 음영 trace 추가
6. geometry2d이면 points, segments, lines, circles를 각각 trace로 변환
7. 3D 계열이면 surface3d, scatter3d 등 Plotly 3D trace로 변환
8. Plotly.newPlot으로 화면에 렌더링

이 구조의 장점은 그래프 종류가 늘어나도 전체 앱을 다시 만들 필요가 없다는 점입니다. 새로운 `kind`와 그 렌더러만 추가하면 됩니다. 반대로 시가 정해진 형식을 벗어나면 화면이 불안정해지므로, `api/coach.js`의 튜터 기준에서 JSON 구조와 예시를 강하게 고정했습니다.

12. 사진 입력과 오답 기록

학생이 수학 문제를 질문할 때 긴 수식과 그림을 매번 정확히 타이핑하기는 어렵습니다. 그래서 사진 입력을 넣었습니다. 다만 원본 사진을 그대로 보내면 요청이 느려지거나 실패할 수 있으므로 브라우저에서 먼저 JPEG로 압축하고, 서버에서도 이미지 크기 상한을 확인합니다.

오답 기록은 서버 데이터베이스가 아니라 같은 기기의 브라우저 안에 저장했습니다. 전달용 앱에서 회원가입과 데이터베이스까지 요구하면 배포 난도가 올라가고, 개인정보 관리도 필요해집니다. 그래서 초기 버전에서는 `localStorage`에 최근 실수 유형만 저장해 “이 기기에서 자주 나온 실수”를 참고하도록 했습니다.

기능	설계 이유	구현 방식
사진 입력	긴 수식, 그림, 분수, 루트를 직접 타이핑하는 부담을 줄입니다.	브라우저에서 JPEG로 압축하고 서버에서 크기 상한을 확인합니다.
대화 이어가기	한 번에 풀어 주지 않고 학생의 다음 시도에 맞춰 이어 갑니다.	<code>convo</code> 배열에 학생/코치 메시지를 누적합니다.
오답 기록	계산, 개념, 조건, 식 세우기, 표기, 검산 실수를 구분합니다.	<code>localStorage</code> 에 최근 실수를 저장하고 다음 코칭에 참고합니다.
질문 한도	공개 배포에서 무료 사용량이 빨리 소진되는 것을 줄입니다.	IP당 시간당 요청 수를 제한하고 실패 요청은 되돌립니다.

13. 일부러 넣지 않은 것

개발 과정에서는 기능을 무조건 늘리는 것보다, 전달받은 사람이 실제로 배포하고 설명할 수 있는 범위를 유지하는 것이 중요했습니다. 그래서 회원가입, 서버 데이터베이스, 선생님용 관리 화면, 완성 풀이 자동 출력 같은 기능은 일부러 넣지 않았습니다.

넣지 않은 것	이유	나중에 확장한다면
회원가입	개인정보 관리와 초기 배포 난도가 올라갑니다.	장기 오답 관리가 필요할 때 추가할 수 있습니다.
서버 DB	설정해야 할 항목이 많아져 전달성이 떨어집니다.	반별 통계나 선생님 화면을 만들 때 연결할 수 있습니다.
완성 풀이 자동 출력	학생이 다시 받아 적는 흐름으로 돌아갈 수 있습니다.	학습 모드와 해설 모드를 분리해 제한적으로 넣을 수 있습니다.
모든 문제 그래프화	의미 없는 그림은 오히려 수학 판단을 흐립니다.	그래프가 도움 되는 단원에만 템플릿을 추가할 수 있습니다.

14. 최종 파일 구조

파일	역할	설명 포인트
index.html	화면 뼈대	문제 사진, 막힌 지점, 결과, 그래프, 이어가기 입력칸을 배치합니다.
style.css	화면 디자인	모바일과 데스크톱에서 읽기 쉬운 학습 UI를 만듭니다.
app.js	브라우저 로직	사진 압축, 요청, 코칭 렌더링, 그래프, 실수 노트, PDF 저장을 처리합니다.
api/coach.js	서버리스 API	Gemini API 키를 숨기고, 튜터 기준과 그래프 결정 기준을 적용합니다.
README.md	전달 요약	앱 의도, 배포 흐름, 그래프 분류 기준을 빠르게 확인하게 합니다.
사용_가이드.pdf	배포 안내	GitHub와 Vercel 배포 과정을 따라 하기 쉽게 정리합니다.
앱_구조_설명서.pdf	탐구·설계 설명	아이디어, SW 구조, 그래프 로직, 한계와 확장 방향을 설명합니다.

15. 설명용 핵심 문장

고3 수학 학습 부진에서 보이는 수식 불안과 수동적 해설 학습을 줄이기 위해, 정답보다 다음 한 걸음을 먼저 제시하고 수식·그래프·오답 기록을 연결한 웹앱을 설계했다. 그래프는 단순 장식이 아니라, 학생의 막힌 지점과 수학 대상에 따라 `function2d`, `geometry2d`, `implicit2d`, 3D 계열, `none` 중 하나를 고르는 표현 로직으로 설계했다.

이 문서는 앱 구조 설명을 위한 전달용 자료이며, 실제 API 키나 개인정보는 포함하지 않습니다.